

Convolutional Networks

Convolution Filters

Introduction to Deep Learning · Class 23

Outline

Why Not a Dense Layer

Invariance and Equivariance

The Operation

Channels, Layers and Receptive Fields

Two Dimensions

Changing Resolution

Putting It Together

Why Not a Dense Layer

Building the symmetry in, rather than learning it

Class 18 listed four ways to obtain an invariance and said the fourth was to build it into the architecture. This class carries that out.

Route to an invariance	What it costs	Class
pre-processing	may discard useful information	18
a penalty on output change	only small transformations	18
augmentation	more data, more compute	19
the architecture	the symmetry must be known	today

Today is about one layer, not about any network. What you build with it is Class 24; how deep to stack it, and what the filters end up detecting, is Class 25.

Notation for today

x, z input and output of a layer

w the **kernel**, of size k

i, j spatial positions

c, c' input and output channels

s stride

d dilation rate

p padding width

r_l receptive field after l layers

- Bishop says *filter* and Prince says *kernel* for the same object; this deck says kernel
- What both books call convolution is, strictly, **cross-correlation**: the kernel is not flipped. The convention is universal and harmless, since the weights are learned either way

Prince (2023), Eq. 10.3 and its footnote; Bishop & Bishop (2024), Eq. 10.1.

Images are too big, and not unstructured

Proposition 1 (The cost of a dense first layer)

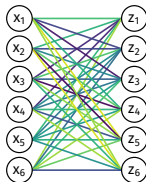
A colour image of $10^3 \times 10^3$ pixels has 3×10^6 inputs. A fully connected first layer with 1,000 hidden units therefore has 3×10^9 weights — in the first layer alone. A convolutional layer with 64 channels and a 3×3 kernel has $3 \times 3 \times 3 \times 64 + 64 = 1,792$, whatever the image size.

- The dense layer also has to **learn** that a cat is still a cat one pixel to the left, from examples
- And it treats the pixels as unstructured: permute them and the layer is unchanged, while the image is destroyed
- Nearby pixels are strongly correlated, and that is prior knowledge worth building in

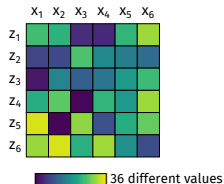
Bishop & Bishop (2024), §10.1.1 and §10.2; Prince (2023), §10.2.4.

The same statement, drawn

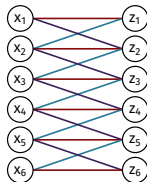
(a) fully connected



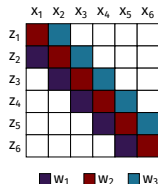
(b) its weight matrix: 36 free



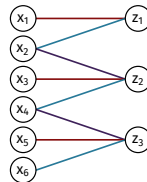
(c) kernel of three



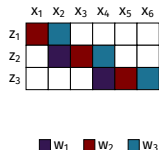
(d) banded and tied: 3 free



(e) kernel of three, stride two



(f) every other row: 3 free



(a–b) a fully connected layer on six inputs: every one of the 36 weights is free. (c–d) a convolutional layer with kernel three: most entries are zero and the rest repeat, so three numbers describe the whole matrix. (e–f) the same with stride two. Prince (2023), Fig. 10.4.

Invariance and Equivariance

What we want of the output

Definition 2 (Invariance and equivariance)

For a transformation $t[\cdot]$ of the input, a function f is *invariant* if

$$f[t[\mathbf{x}]] = f[\mathbf{x}], \quad \text{and } \textit{equivariant} \text{ if } f[t[\mathbf{x}]] = t[f[\mathbf{x}]].$$

- Classify an image: translate it and the label must not move — invariance
- Segment an image: translate it and the mask must move with it — equivariance
- A network gets invariance by making its *layers* equivariant and then discarding position at the end

Prince (2023), Eqs. 10.1–10.2, Fig. 10.1; Bishop & Bishop (2024), §10.2.2.

Convolution is translation-equivariant

Proposition 3 (Equivariance)

Let T_a shift a signal by a positions. A convolution $z_i = \sum_j w_j x_{i+j}$ satisfies

$$\text{conv}(T_a \mathbf{x}) = T_a \text{conv}(\mathbf{x}) \quad \text{for every } a,$$

up to the treatment of the boundary. Shifting the input shifts the output by the same amount and changes nothing else.

- This is the property that lets a feature learned at one position be recognised at every other, without seeing it there
- It is a consequence of using the *same* weights at every position, and of nothing else

Bishop & Bishop (2024), §10.2.2, Fig. 10.2; Prince (2023), §10.2.1.

Proof

- Write $y_i = (T_a \mathbf{x})_i = x_{i-a}$: the input, shifted by a .
- Apply the convolution to it: $\text{conv}(\mathbf{y})_i = \sum_j w_j y_{i+j} = \sum_j w_j x_{i-a+j}$.
- The right-hand side is exactly what $\text{conv}(\mathbf{x})$ evaluates to at position $i - a$.
- And $\text{conv}(\mathbf{x})_{i-a}$ is the i -th entry of $T_a \text{conv}(\mathbf{x})$, which is the claim. □
- Every step used the fact that w_j does not depend on i . A layer whose weights varied with position would fail at step 2.

Sharing the weights is not a trick to save memory. It is what equivariance requires, and the saving is a consequence.

Proof

- Write $y_i = (T_a \mathbf{x})_i = x_{i-a}$: the input, shifted by a .
- **Apply the convolution to it:** $\text{conv}(\mathbf{y})_i = \sum_j w_j y_{i+j} = \sum_j w_j x_{i-a+j}$.
- The right-hand side is exactly what $\text{conv}(\mathbf{x})$ evaluates to at position $i - a$.
- And $\text{conv}(\mathbf{x})_{i-a}$ is the i -th entry of $T_a \text{conv}(\mathbf{x})$, which is the claim. □
- Every step used the fact that w_j does not depend on i . A layer whose weights varied with position would fail at step 2.

Sharing the weights is not a trick to save memory. It is what equivariance requires, and the saving is a consequence.

Proof

- Write $y_i = (T_a \mathbf{x})_i = x_{i-a}$: the input, shifted by a .
- Apply the convolution to it: $\text{conv}(\mathbf{y})_i = \sum_j w_j y_{i+j} = \sum_j w_j x_{i-a+j}$.
- **The right-hand side is exactly what $\text{conv}(\mathbf{x})$ evaluates to at position $i - a$.**
- And $\text{conv}(\mathbf{x})_{i-a}$ is the i -th entry of $T_a \text{conv}(\mathbf{x})$, which is the claim. □
- Every step used the fact that w_j does not depend on i . A layer whose weights varied with position would fail at step 2.

Sharing the weights is not a trick to save memory. It is what equivariance requires, and the saving is a consequence.

Proof

- Write $y_i = (T_a \mathbf{x})_i = x_{i-a}$: the input, shifted by a .
- Apply the convolution to it: $\text{conv}(\mathbf{y})_i = \sum_j w_j y_{i+j} = \sum_j w_j x_{i-a+j}$.
- The right-hand side is exactly what $\text{conv}(\mathbf{x})$ evaluates to at position $i - a$.
- **And $\text{conv}(\mathbf{x})_{i-a}$ is the i -th entry of $T_a \text{conv}(\mathbf{x})$, which is the claim.** □
- Every step used the fact that w_j does not depend on i . A layer whose weights varied with position would fail at step 2.

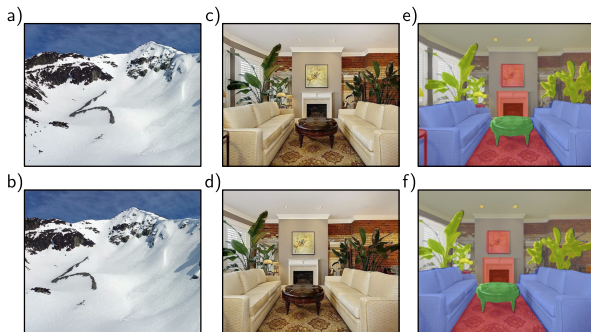
Sharing the weights is not a trick to save memory. It is what equivariance requires, and the saving is a consequence.

Proof

- Write $y_i = (T_a \mathbf{x})_i = x_{i-a}$: the input, shifted by a .
- Apply the convolution to it: $\text{conv}(\mathbf{y})_i = \sum_j w_j y_{i+j} = \sum_j w_j x_{i-a+j}$.
- The right-hand side is exactly what $\text{conv}(\mathbf{x})$ evaluates to at position $i - a$.
- And $\text{conv}(\mathbf{x})_{i-a}$ is the i -th entry of $T_a \text{conv}(\mathbf{x})$, which is the claim. □
- Every step used the fact that w_j does not depend on i . A layer whose weights varied with position would fail at step 2.

Sharing the weights is not a trick to save memory. It is what equivariance requires, and the saving is a consequence.

Equivariance, on an image



(a–b) classification: the horizontal shift must leave the label “mountain” unchanged — invariance. (c–f) segmentation: when the image is translated, the per-pixel output must translate with it — equivariance.

Prince (2023), Fig. 10.1.

The Operation

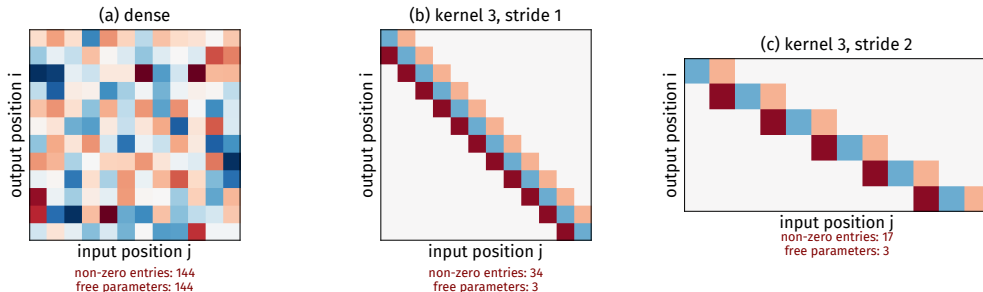
One dimension, one kernel

$$z_i = \sum_{j=1}^k w_j x_{i+j-\lceil k/2 \rceil}, \quad h_i = a \left[\beta + \sum_{j=1}^k w_j x_{i+j-2} \right] \quad (k=3).$$

- Each output reads a window of k inputs. The same k weights are used at **every** position — this is the parameter sharing of Class 18, in its canonical form
- A convolutional layer adds a bias and an activation, exactly as a dense layer does
- Compare a dense layer, $h_i = a[\beta_i + \sum_{j=1}^D \Omega_{ij} x_j]$: D^2 weights and D biases against k weights and one bias

Prince (2023), Eqs. 10.3–10.5, Figs. 10.2, 10.4.

A convolution is a dense layer with most of it deleted



The actual matrix each layer applies, on twelve inputs. (a) dense: 144 free numbers. (b) kernel three: 34 non-zeros but only **three** free numbers, because the colours repeat down the band. (c) stride two removes every other row. Sparsity alone would leave $3n$ free weights; tying them is what leaves three.

Three knobs, and one formula

Proposition 4 (Output length)

A 1-D convolution with kernel size k , stride s , dilation d and zero padding p on an input of length n produces

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - d(k - 1) - 1}{s} \right\rfloor + 1.$$

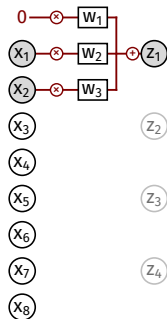
The dilated kernel spans $d(k - 1) + 1$ positions while using only k weights.

- **Padding** decides whether the representation shrinks at the edges; *valid* padding invents nothing but loses size
- **Stride** subsamples the output; **dilation** widens the reach without buying more weights

Prince (2023), §10.2.2–10.2.3, Fig. 10.3; Bishop & Bishop (2024), §10.2.3–10.2.4; Dumoulin & Visin (2016).

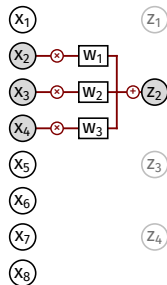
The three knobs, drawn

(a) size 3, stride 2



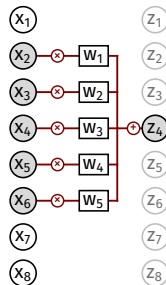
$k=3, s=2, d=1$

(b) size 3, stride 2



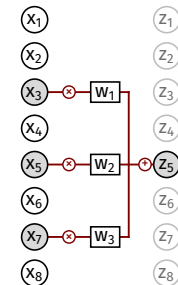
$k=3, s=2, d=1$

(c) size 5, stride 1



$k=5, s=1, d=1$

(d) size 3, dilation 2



$k=3, s=1, d=2$

Eight inputs each time; shaded inputs are the ones the shaded output reads, through the weights w_j . (a–b) stride two evaluates the kernel at every other position, and z_1 reads a zero pad. (c) a kernel of five reads more and costs more weights. (d) a dilated kernel reaches as far as a kernel of five while paying for three.

Prince (2023), Fig. 10.3.

What one kernel responds to

Proposition 5 (The kernel is what it detects)

*For a unit $z = \text{ReLU}(\mathbf{w}^T \mathbf{x} + w_0)$ acting on a patch $\mathbf{x}$ of fixed norm, the patch maximising the response is $\mathbf{x} = \gamma \mathbf{w}$ for some $\gamma > 0$. So the unit fires hardest on the patch that *looks like its own kernel*, and the bias sets how good a match is good enough.*

- One line, by Cauchy–Schwarz: $\mathbf{w}^T \mathbf{x} \leq \|\mathbf{w}\| \|\mathbf{x}\|$ with equality only when $\mathbf{x}$ is parallel to $\mathbf{w}$
- This is why a kernel can be drawn as a small image and read as a template
- What templates a trained network settles on is Class 25

Bishop & Bishop (2024), §10.2.1, Eq. 10.1, Exercise 10.1.

Channels, Layers and Receptive Fields

One kernel is not enough

$$h_i^{c'} = a\left[\beta_{c'} + \sum_{c=1}^{C_{\text{in}}} \sum_{j=1}^k \Omega_{c, c', j} x_{i+j-2}^c\right], \quad \Omega \in \mathbb{R}^{C_{\text{in}} \times C_{\text{out}} \times k}.$$

- A single convolution averages and then clips, so information is lost. Several are run in parallel, and each output is a **channel** or **feature map**
- Every output channel reads *all* input channels over k positions, so the layer has $C_{\text{in}}C_{\text{out}}k$ weights and C_{out} biases
- Still independent of the length of the input

Prince (2023), §10.2.5, Fig. 10.5.

How far back a unit can see

Definition 6 (Receptive field)

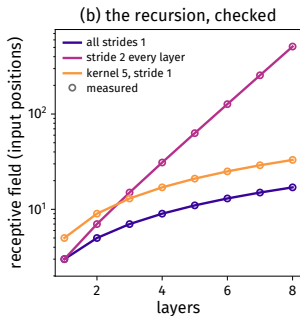
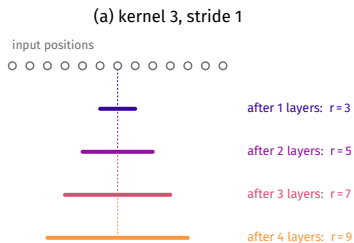
The *receptive field* of a hidden unit is the region of the original input that feeds into it. A unit that has not seen a pixel cannot depend on it, so it is the budget within which any feature must fit.

Layers of kernel 3	all strides 1	stride 2 throughout
one	3	3
two	5	5
three	7	9
four	9	17

- At stride one each layer adds $k - 1$ positions; with stride two the increments double as well

Prince (2023), §10.2.6, Fig. 10.6.

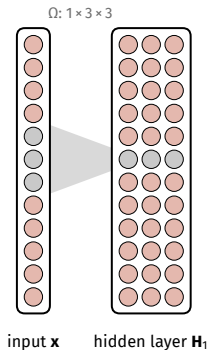
The field, measured



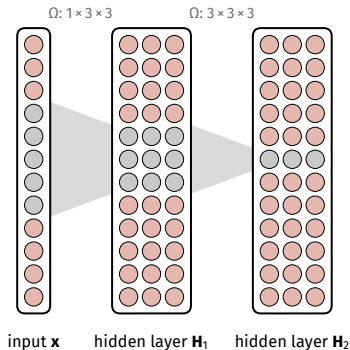
(a) the span reaching one unit after one to four layers of kernel three and stride one: 3, 5, 7, 9. (b) the same quantity against depth for three schedules, measured by propagating a single one back through the stack.

How far back a unit can see, drawn

(a) one layer: three inputs



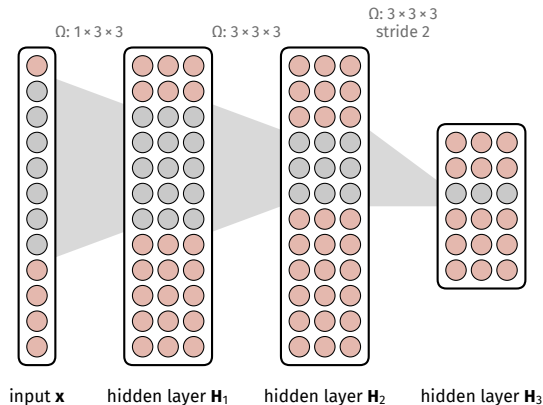
(b) two layers: five inputs



Each layer is a column of units, three channels wide after the input; grey units lie in the receptive field of the shaded row on the right, and the spotlight shows which units of one layer reach the next. (a) a unit in the first hidden layer sees three inputs. (b) one in the second sees five, because each of its three inputs saw three.

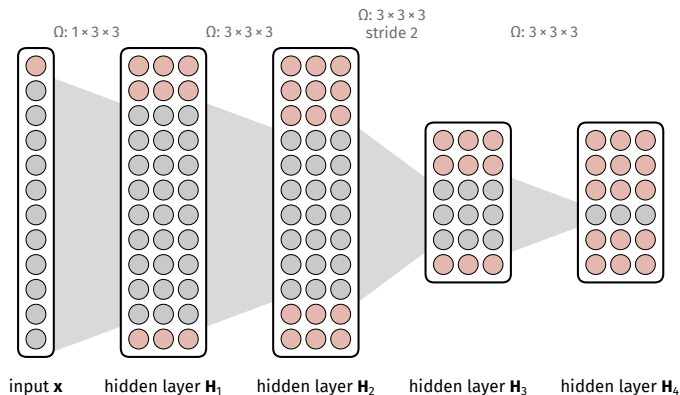
Prince (2023), Fig. 10.6.

How far back a unit can see, drawn



(c) a third layer with stride two halves the number of units and doubles the step of every kernel after it: a unit there reaches seven inputs. Prince (2023), Fig. 10.6.

How far back a unit can see, drawn



(d) by the fourth layer the field covers eleven of the twelve inputs. Every field here is computed by propagating the shaded row back through the stack, one layer at a time.

Prince (2023), Fig. 10.6.

Why depth beats a wider kernel

Layer stack	Receptive field	Weights (C channels)	Nonlinearities
one 5×5	5	$25 C^2$	1
two 3×3	5	$18 C^2$	2
one 7×7	7	$49 C^2$	1
three 3×3	7	$27 C^2$	3

- Same reach, fewer weights, and more nonlinearity between the input and the output
- This is why 3×3 became the default kernel, and it is an argument about the *stack*, not the layer

Which stacks were actually adopted, and what their filters end up detecting, is Class 25.

Two Dimensions

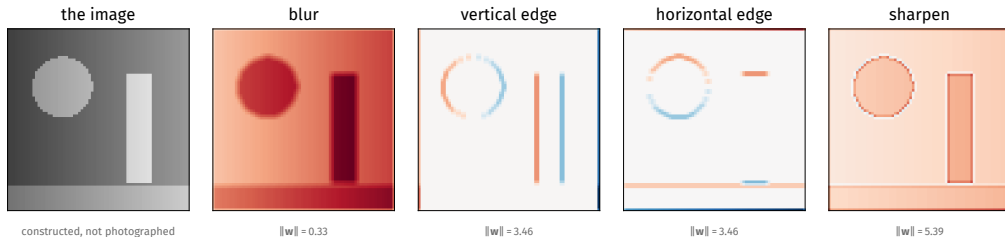
The same operation, one index more

$$h_{i,j}^{c'} = a \left[\beta_{c'} + \sum_{c=1}^{C_{\text{in}}} \sum_{u=1}^k \sum_{v=1}^k \Omega_{c,c',u,v} x_{i+u-2, j+v-2}^c \right]$$

- Nothing conceptual changes: the window is a square, the weights are still shared across positions, and the layer is still equivariant — now to translations in two directions
- Parameters: $k^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}$. For $k = 3$, $C_{\text{in}} = C_{\text{out}} = 64$: 36,928, whatever the image size
- A 1×1 convolution mixes channels and nothing else, which is how the channel count is changed cheaply

Prince (2023), §10.3, Eq. 10.6, Figs. 10.8–10.10; Bishop & Bishop (2024), §10.2.5.

What a 3×3 kernel does



The same nine weights applied at every position of a constructed scene. The edge kernels respond only where the intensity changes, and only in the direction they were built for — which is Proposition 5 seen from the other side. These four are hand-chosen; a network learns its own.

What a 3×3 kernel does, on an image



(a)



(b)



(c)

(a) the original image. (b) the output of a filter that responds to vertical edges. (c) the output of one that responds to horizontal edges. The same small kernel is applied at every position. Bishop & Bishop (2024),

Fig. 10.4.

Changing Resolution

Down, and back up

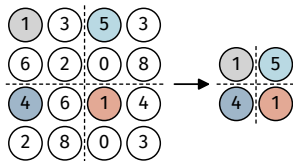
Downsampling	Keeps	Note
max pooling	the strongest response in each window	no parameters
mean pooling	the average of each window	no parameters
strided convolution	a learned summary	has parameters
Upsampling	Fills in with	Note
duplication	the nearest value	no parameters
zero insertion	zeros, then a convolution	<i>transposed</i> convolution
max unpooling	the value at the remembered argmax	needs the indices
bilinear	a linear interpolation	no parameters

- Pooling trades **where** for **whether**: tolerance to small shifts, bought with the ability to localise

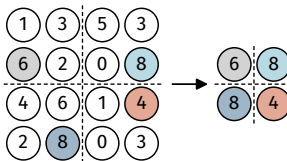
Prince (2023), §10.4; Bishop & Bishop (2024), §10.2.6.

Resolution, drawn

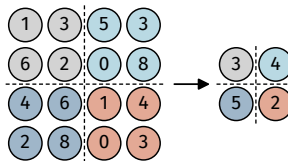
(a) sub-sampling



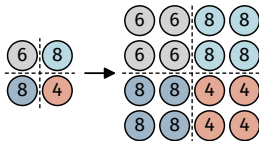
(b) max pooling



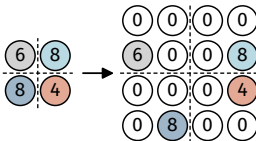
(c) mean pooling



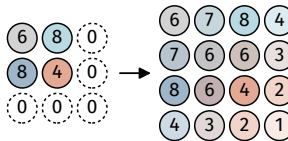
(d) duplication



(e) max unpooling



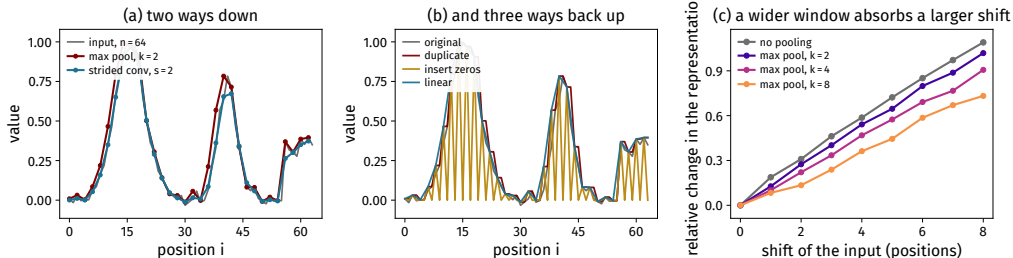
(f) bilinear interpolation



A 4×4 array, each block in its own tint; every number is computed. Down: (a) sub-sampling keeps one entry per block, (b) max pooling the largest, (c) mean pooling the average. Back up from (b): (d) duplication, (e) max unpooling to the remembered positions, zeros elsewhere, (f) bilinear interpolation, which blends the numbers and the tints alike.

Prince (2023), Figs. 10.11 and 10.12.

Resolution, measured



(a) two ways to halve a signal. (b) three ways to put it back: none recovers what pooling discarded, which is why an architecture that needs the detail keeps a copy of it. (c) translating the input by one position changes the raw signal by 14% and an eight-wide pooled signal by 6%.

Putting It Together

The layer, in one table

Choice	What it controls	Cost
kernel size k	how much context one unit reads	k or k^2 weights per channel pair
stride s	the output resolution	none; discards positions
dilation d	the reach, without more weights	gaps in what is read
padding p	whether the edges survive	invents values, or loses size
channels C_{out}	how many features per position	linear in C_{out}
depth L	the receptive field, and nonlinearity	linear in L
pooling	tolerance to small shifts	the ability to localise

Everything above is a statement about the layer. Not one of these choices depends on the size of the image, which is exactly the property the dense layer lacked.

Takeaways

1. A dense first layer on a 1000^2 colour image needs 3×10^9 weights; a 3×3 convolution with 64 channels needs 1,792, whatever the image size (Proposition 1)
2. Classification wants invariance and segmentation wants equivariance; equivariant layers plus a final discard of position give both (Definition 2)
3. Convolution is translation-equivariant, which is what lets a feature learned at one position be found at every other (Proposition 3)
4. Weight sharing is what equivariance forces; the parameter saving is a consequence, not the motive
5. $n_{\text{out}} = \lfloor (n + 2p - d(k - 1) - 1)/s \rfloor + 1$ covers padding, stride and dilation at once (Proposition 4)
6. A unit responds most to the patch that looks like its own kernel (Proposition 5)
7. The receptive field grows with depth, slowly at stride one and far faster once strides are introduced (Definition 6)
8. Two 3×3 layers reach as far as one 5×5 with fewer weights and one more nonlinearity

Class 24 asks what to build out of this layer: one label, a set of boxes, or one label per pixel.

References i

1. C. M. Bishop & H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. §10.1–10.1.1; §10.2.1–10.2.6 (Eq. 10.1, Figs. 10.1–10.7, Exercise 10.1).
2. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. §10.1–10.4 (Eqs. 10.1–10.6, Figs. 10.1–10.13).
3. K. Fukushima, “Neocognitron: a self-organizing neural network model”, *Biological Cybernetics* 36, 1980.
4. Y. LeCun, B. Boser, J. S. Denker et al., “Backpropagation applied to handwritten zip code recognition”, *Neural Computation* 1(4), 1989.
5. Y. LeCun, L. Bottou, Y. Bengio & P. Haffner, “Gradient-based learning applied to document recognition”, *Proc. IEEE* 86(11), 1998.
6. F. Yu & V. Koltun, “Multi-scale context aggregation by dilated convolutions”, ICLR 2016.
7. V. Dumoulin & F. Visin, “A guide to convolution arithmetic for deep learning”, arXiv:1603.07285, 2016.
8. W. Luo, Y. Li, R. Urtasun & R. Zemel, “Understanding the effective receptive field in deep convolutional neural networks”, NIPS 2016.

Questions?